

Universe Reinsurance Platform

Runbook: Fixing the Git Clone Authentication Error

"Password authentication is not supported for Git operations"

Version 1.0 — July 2026 — Confidential — For Internal IT Use Only

Companion to the Universe Deployment Specification v2.0 (§6.1, §12.1)

1. The Error

Attempting to clone the application repository onto the deployment server fails like this:

```
gaudire@sr-urp:~$ sudo -u universe -H git clone https://github.com/iwadaya/universe.git /opt/universe
Cloning into '/opt/universe'...
Username for 'https://github.com': iwadaya
Password for 'https://iwadaya@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/iwadaya/universe.git/'
```

1.1 Why It Happens (two separate problems)

Problem	Explanation
1. Wrong repository URL	The repository has moved. <code>github.com/iwadaya/universe</code> (and the older <code>iwadaya/claude_universe</code>) no longer exist. The current private repository is <code>github.com/Darchville-Analytics/modelling_tool</code> .
2. Password typed at the prompt	GitHub removed account-password authentication for Git operations in August 2021. The "Password" prompt actually expects a Personal Access Token (PAT) . Typing your GitHub account password will always fail with exactly this message — even against a valid URL.

Note: because the old URL is dead, even a valid token used against `iwadaya/universe.git` would still fail. Both problems must be fixed together.

2. Fix Overview

1. Create a fine-grained Personal Access Token on GitHub (§3) — or set up an SSH deploy key instead (§7).
2. Make sure the target directory `/opt/universe` is empty or removed, preserving `.env` and uploads if a previous install exists (§4).
3. Store the credential for the `universe` user and clone from the correct URL (§5).
4. Verify the clone and continue the deployment (§6).

Steps assume you are logged in as your admin account (e.g. `gaudire`) with sudo rights, and that the application runs as the dedicated non-root user `universe`.

3. Step 1 — Create a Fine-Grained Personal Access Token

On **github.com**, logged in as an account that has access to the `Darchville-Analytics/modelling_tool` repository:

1. Click your avatar (top-right) → **Settings**.

2. In the left sidebar, scroll to the bottom → **Developer settings**.
3. Choose **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
4. Fill in:
 - **Token name:** e.g. `universe-deploy-sr-urp` (name it after the server so it's identifiable later)
 - **Expiration:** 90 days (matches the rotation policy in the deployment spec §9.2)
 - **Resource owner:** **Darchville-Analytics** — not your personal account. If the org doesn't appear in the dropdown, your account is not a member; ask an org admin to invite you first.
 - **Repository access:** *Only select repositories* → `modelling_tool`
 - **Permissions** → **Repository permissions:** **Contents: Read-only**. Nothing else is needed for a deploy box.
5. Click **Generate token** and **copy it immediately** (it starts with `github_pat_...`). It is shown only once — if you lose it, regenerate.

Org approval: if Darchville-Analytics requires fine-grained token approval, the token stays "pending" until an org admin approves it under *Org Settings* → *Personal access tokens*. A pending token behaves exactly like a bad password — if your clone still fails after this runbook, check approval status first.

4. Step 2 — Make Sure /opt/universe Is Empty or Removed

`git clone` refuses to clone into a directory that already contains files (`fatal: destination path '/opt/universe' already exists and is not an empty directory`). Check what is there before deleting anything.

4.1 Check the directory

```
ls -la /opt/universe
```

Result	What to do
No such file or directory	Nothing to clean up — go straight to Step 3 (§5).
Directory exists but is empty (only <code>.</code> and <code>..</code> listed)	Also fine — git can clone into an empty directory. Go to Step 3 (§5).
Directory has files in it	Continue with §4.2 before removing.

A clone attempt that failed at authentication (as in §1) normally leaves nothing behind — git cleans up after itself — so on this server the directory is most likely missing or empty.

4.2 If files exist: preserve anything irreplaceable first

Before running `rm -rf` , check for the two things that cannot be recovered from GitHub: the server's `.env` file (secrets and configuration) and the `uploads` directory (slips and documents uploaded by users). Everything else in the directory is just code that the fresh clone replaces.

```
# Check whether either exists
ls -la /opt/universe/.env
ls -la /opt/universe/uploads
```

If either exists, back it up before deleting:

```
sudo cp /opt/universe/.env /root/universe.env.bak
sudo cp -r /opt/universe/uploads /root/universe-uploads.bak
```

4.3 Remove the directory

```
sudo rm -rf /opt/universe
```

Careful with this command. Double-check the path before pressing Enter — `rm -rf` deletes recursively and permanently, with no confirmation and no recycle bin.

5. Step 3 — Store the Credential and Clone from the Correct URL

5.1 Enable the credential store for the universe user

This makes git remember the token after the first use, so `git pull` and `deploy.sh` never prompt again:

```
sudo -u universe -H git config --global credential.helper store
```

5.2 Clone

```
sudo -u universe -H git clone \
    https://github.com/Darchville-Analytics/modelling_tool.git /opt/universe
```

At the prompts:

Prompt	Enter
Username for 'https://github.com':	Your GitHub username (the account that owns the token)
Password for 'https://...':	Paste the fine-grained token (<code>github_pat_...</code>) — NOT your GitHub account password. Nothing is echoed to the screen while you paste; that is normal — just press Enter.

The token is saved to `/home/universe/.git-credentials` with mode 600 (readable only by the `universe` user).

5.3 If you backed up .env / uploads in §4.2, restore them now

```
sudo cp /root/universe.env.bak /opt/universe/.env
sudo cp -r /root/universe-uploads.bak /opt/universe/uploads
sudo chown -R universe:universe /opt/universe
sudo chmod 600 /opt/universe/.env
```

5.4 Alternative: existing checkout that only has the wrong URL

If a working install already exists and the only problem is that its remote still points at the old repository, do not delete anything — just repoint the remote:

```
cd /opt/universe
sudo -u universe -H git remote set-url origin \
    https://github.com/Darchville-Analytics/modelling_tool.git
sudo -u universe -H git fetch origin
sudo -u universe -H git pull origin main
```

6. Step 4 — Verify and Continue the Deployment

```
# Confirm the clone landed and the remote is correct
ls /opt/universe
cd /opt/universe && git remote -v
# expect: origin https://github.com/Darchville-Analytics/modelling_tool.git (fetch/push)

# Confirm authenticated pulls work without prompting
sudo -u universe -H git -C /opt/universe pull origin main
```

Success looks like: `git remote -v` shows the `Darchville-Analytics/modelling_tool` URL, and `git pull` completes (or says `Already up to date.`) without asking for a username or password.

Then continue with the Deployment Specification v2.0 from §6.2 onwards:

```
cd /opt/universe
npm run install:all
npm run build
cp .env.example .env && nano .env && chmod 600 .env # skip if you restored a backed-up .env
npm run migrate:up --prefix server
pm2 start ecosystem.config.cjs && pm2 save
```

7. Alternative Method — SSH Deploy Key (No Token on the Server)

If you prefer not to manage a PAT on the server (nothing to rotate every 90 days), use a read-only SSH deploy key instead. Requires outbound access to `github.com` on port 22.

1. Generate a key pair as the `universe` user:

```
sudo -u universe -H ssh-keygen -t ed25519 -C "universe-deploy" -N "" \
-f /home/universe/.ssh/id_ed25519
```

2. Print the public key and copy it:

```
sudo -u universe -H cat /home/universe/.ssh/id_ed25519.pub
```

3. On GitHub: open `Darchville-Analytics/modelling_tool` → **Settings** → **Deploy keys** → **Add deploy key**. Paste the key, name it after the server, and leave **"Allow write access"** **unticked** (a deploy box only needs to read).

4. Clone over SSH (directory rules from §4 still apply):

```
sudo -u universe -H git clone \
git@github.com:Darchville-Analytics/modelling_tool.git /opt/universe
```

On the first connection, type `yes` when asked to trust `github.com`'s host key.

A deploy key is tied to this one repository and this one server — it grants no access to anything else, which is why it's the tidiest option for shared servers.

8. Still Failing? Checklist

Symptom	Check / fix
Same "Password authentication is not supported" after entering the token	You pasted the account password out of habit, or the token has a typo. Paste the <code>github_pat_...</code> value exactly; nothing echoes while pasting.
<code>remote: Repository not found</code>	URL typo, the token's <i>Repository access</i> doesn't include <code>modelling_tool</code> , the resource owner was set to your personal account instead of Darchville-Analytics , or your GitHub account lacks access to the repo. GitHub deliberately reports private-repo permission failures as "not found".
Token worked before, fails now	Token expired (90-day default) or was revoked. Generate a new one, then refresh the stored credential: <code>sudo -u universe -H rm /home/universe/.git-credentials</code> and pull once to re-enter it.
Token generated but clone fails immediately	Org token approval pending — ask a Darchville-Analytics admin to approve it (§3 note).
<code>destination path '/opt/universe'</code> already exists and is not an empty directory	Directory not cleaned up — redo §4 (back up <code>.env</code> /uploads, then <code>sudo rm -rf /opt/universe</code>), or use §5.4 if the checkout is otherwise healthy.
Prompted for credentials on every pull	Credential helper not set for the right user. Re-run <code>sudo -u universe -H git config --global credential.helper store</code> (the <code>-H</code> matters — it makes git write to <code>/home/universe</code> , not your admin home) and pull once.
SSH clone: <code>Permission denied (publickey)</code>	The deploy key isn't added to the repo, or the clone is running as a different user than the one whose key you generated. Test with <code>sudo -u universe -H ssh -T git@github.com</code> .
Connection timeouts to github.com	Corporate firewall/proxy: HTTPS needs outbound 443, SSH needs outbound 22. Ask network team to allow the server to reach <code>github.com</code> .

9. Security Notes

- Never use a GitHub **account password** or a classic full-scope token for deployments — scope the credential to one repository, read-only.
- Rotate the PAT every 90 days (set a calendar reminder for the expiry date); deploy keys don't expire but should be removed when a server is decommissioned.
- The stored credential file (`/home/universe/.git-credentials`) contains the token in plain text with mode 600 — do not loosen its permissions or copy it elsewhere.
- Avoid embedding the token in the remote URL (`https://TOKEN@github.com/...`) as the old v1.0 manual suggested — it ends up readable in `.git/config` and in shell history.

- Delete the temporary backups in `/root` (`universe.env.bak` , `universe-uploads.bak`) once the restored files are confirmed working.